System Design for Beginners (2026)

Photo-Sharing App bana ke seekho — har concept Roman Urdu mein, simple examples ke saath. Video ke topics aur examples ko detail mein cover kiya gaya hai.

Video Reference: "System Design for Beginners (2026)"

Concept: Ek real photo-sharing app (jaise Instagram) step-by-step design karke system design ke saare concepts seekhna.

TABLE OF CONTENTS

01. Client-Server Model — Internet Kaise Kaam Karta Hai
02. IP Address — Ghar Ka Pata
03. DNS — Phone Book of Internet
04. HTTP / HTTPS — Baat Cheet Ka Tarika
05. APIs (REST) — System Ka Darwaza
06. API Gateway — Building Ka Receptionist
07. Proxy & Reverse Proxy — Beech Ka Aadmi
08. Load Balancer — Traffic Warden
09. Databases — SQL vs NoSQL
10. Blob Storage — Photos/Videos Ka Godown
11. CDN — Content Delivery Network
12. Caching — Speed Ka Raaz
13. Message Queues — Background Kaam
14. Scaling — Horizontal vs Vertical
15. Replication — Database Ki Photocopy
16. Sharding — Data Ko Todh Do
17. CAP Theorem — Teen Mein Se Do Chuno
18. Microservices vs Monolith
19. Rate Limiting — Abuse Se Bachaao
20. WebSockets — Real-Time Connection

21. Trade-offs & Summary

SECTION 01

Client-Server Model — Internet Kaise Kaam Karta Hai

Jab bhi tu browser mein kuch type karta hai ya app kholata hai, toh tera device (phone/laptop) ek **Client** hai aur jahan se data aata hai woh **Server** hai. Client request bhejta hai, Server response deta hai. Yehi poore internet ki buniyaad hai.

```
Client (Tera Phone) --request--> Server (Computer jo data rakhta hai) --response--> Client
```

EXAMPLE — Photo-Sharing App

Jab tu Instagram kholta hai aur feed scroll karta hai — tera phone (client) Instagram ke server ko bolta hai 'mujhe latest photos bhejo'. Server database se photos nikaal ke tere phone ko bhej deta hai. Jab tu photo upload karta hai — tera phone server ko photo bhejta hai, server usse save karta hai.

KEY POINT: Har web app Client-Server model pe kaam karti hai. Client = jo maangta hai. Server = jo deta hai.

SECTION 02

IP Address — Ghar Ka Pata

Har device (phone, laptop, server) ka internet pe ek unique pata hota hai jisko **IP Address** kehte hain. Jaise tere ghar ka address hota hai taa ke courier wala aa sake — waise hi server ka IP Address hota hai taa ke tera request sahi jagah pohanch sake.

Example: **192.168.1.1** (IPv4) ya **2001:db8::1** (IPv6). Jab tu google.com type karta hai, peeche se tera browser Google ke server ka IP address dhoondh ke uspe request bhejta hai.

EXAMPLE — Photo-Sharing App

Teri photo-sharing app ka server ka IP address hai jaise 34.102.136.180. Jab koi user app kholata hai, uska phone is address pe request bhejta hai. Lekin kisi ko ye number yaad nahi — isliye DNS use hota hai (agla topic).

SECTION 03

DNS — Phone Book of Internet

DNS (Domain Name System) internet ki phone book hai. Ye human-readable naam (jaise google.com) ko machine-readable IP address (jaise 142.250.80.46) mein convert karta hai. Tujhe IP address yaad rakhne ki zaroorat nahi — bas naam type kar, DNS dhoondh lega.

```
Tu type karta hai: photoshare.com --> DNS Server: '142.250.80.46 hai' --> Browser
us IP pe request bhejta hai
```

EXAMPLE — Phone Book Analogy

Socho tere phone mein contacts hain. Tu 'Ammi' search karta hai — phone number khud mil jaata hai. Tujhe number yaad nahi rakhna padta. **DNS = Internet ka Contacts App.** 'photoshare.com' type kiya, DNS ne server ka IP address dhoondh ke de diya.

KEY POINT: DNS ke bina tujhe har website ka IP number yaad rakhna padta. DNS ye kaam automatic karta hai — naam se IP dhoondh deta hai.

SECTION 04

HTTP / HTTPS — Baat Cheet Ka Tarika

HTTP (HyperText Transfer Protocol) woh language hai jismein client aur server baat karte hain. Jaise do insaan Urdu mein baat karte hain — client aur server HTTP mein baat karte hain.

HTTPS mein 'S' ka matlab hai **Secure**. Isme data encrypt ho ke jaata hai — beech mein koi padh nahi sakta. Jaise tala laga ke khat bhejna.

HTTP Methods (tarike):

GET = Data maangna (photos dekhna) | **POST** = Naya data bhejna (photo upload) | **PUT** = Update karna (caption edit) | **DELETE** = Mitana (photo delete)

EXAMPLE — Photo-Sharing App

Jab tu feed scroll karta hai: GET /photos (photos maangna). Jab tu nai photo upload karta hai: POST /photos (nai photo bhejna). Jab tu caption edit karta hai: PUT /photos/123 (photo 123 update karna). HTTPS isliye zaroori hai kyun ke tera password aur private photos beech mein koi na padh sake.

SECTION 05

APIs (REST) — System Ka Darwaza

API (Application Programming Interface) tera system ka darwaza hai. Frontend (ya doosra system) isse baat karta hai. REST API mein har resource ka ek URL hota hai aur HTTP methods use hote hain.

Socho **API ek waiter hai** restaurant mein. Tu (client) waiter ko bolta hai kya chahiye — waiter kitchen (server) mein jaata hai, khaana laata hai. Tu kabhi seedha kitchen mein nahi jaata.

EXAMPLE — Photo-Sharing App ke Endpoints

GET /api/photos — Saari photos lao **POST /api/photos** — Nai photo upload karo **GET /api/photos/123** — Photo #123 ki detail lao **DELETE /api/photos/123** — Photo #123 delete karo **GET /api/users/ali/photos** — Ali ki saari photos lao Ye sab darwaze hain — frontend in URLs pe requests bhejta hai, server jawaab deta hai.

KEY POINT: Achi API simple, predictable, aur consistent honi chahiye. /users, /photos, /comments — clear naming taa ke documentation ke bina samajh aaye.

SECTION 06

API Gateway — Building Ka Receptionist

API Gateway ek single entry point hai jahan se saare requests aate hain. Ye decide karta hai ke konsa request konsi service ko jaaye. Saath mein ye authentication check karta hai, rate limiting lagata hai, aur logging karta hai.

```
Client --> API Gateway --> Photo Service --> User Service --> Notification Service
```

EXAMPLE — Office Building Ka Receptionist

Socho ek bari building hai jismein 10 companies hain. Neeche ek receptionist baitha hai. Tu aata hai aur bolta hai 'mujhe ABC Company se milna hai'. Receptionist check karta hai tera appointment hai ya nahi (authentication), phir tujhe sahi floor pe bhejta hai (routing). **Receptionist = API Gateway. Companies = Different Services.**

KEY POINT: API Gateway ke kaam: Routing (sahi service pe bhejta hai), Authentication (kon hai check karta hai), Rate Limiting (zyada requests rok deta hai), Logging (record rakhta hai).

SECTION 07

Proxy & Reverse Proxy — Beech Ka Aadmi

Forward Proxy	Reverse Proxy
Client ki taraf se kaam karta hai. Jaise VPN — tera request pehle proxy pe jaata hai, phir proxy server se baat karta hai. Server ko nahi pata ke asli request kisne bheji. Privacy aur censorship bypass ke liye use hota hai.	**Server ki taraf se kaam karta hai.** Client ka request pehle reverse proxy pe aata hai, phir woh decide karta hai konse server ko bhejey. Client ko nahi pata ke peeche kitne servers hain. Nginx, Cloudflare — ye sab reverse proxy hain.

EXAMPLE — Photo-Sharing App

Forward Proxy: Tu office mein ho aur Instagram blocked hai. VPN (proxy) use karke access kar liya — office ko nahi pata tu kya access kar raha hai. **Reverse Proxy:** Teri photo app ke peeche 5 servers hain. Reverse proxy (Nginx) bahar se aane wali requests le ke andar ke servers mein baant deta hai. User ko sirf ek address dikhai deta hai.

SECTION 08

Load Balancer — Traffic Warden

Jab tere paas multiple servers hain, toh **Load Balancer** decide karta hai ke konsa request konse server ko jaaye. Iske bina saare requests ek server pe aayenge aur woh crash ho jaayega.

```
1000 Users --> Load Balancer --> Server 1 (333 requests) --> Server 2 (333 requests) --> Server 3 (334 requests)
```

EXAMPLE — Traffic Signal Analogy

Socho ek chowk pe 4 raaste hain. Agar signal na ho toh sab gaariyan ek taraf jaayen gi aur traffic jam. Signal (Load Balancer) decide karta hai konsi gaari (request) konse raaste (server) pe jaaye. Sab balance mein rehta hai. **Photo App mein:** Eid pe 10 lakh log photos upload kar rahe hain. Ek server handle nahi kar sakta. Load balancer 5 servers mein requests baant deta hai — har server 2 lakh handle karta hai.

Load Balancing Algorithms:

Round Robin: Baari baari — request 1 server A ko, request 2 server B ko, request 3 server C ko... **Least Connections:** Jis server pe sabse kam load hai, uspe bhejo. **IP Hash:** Same user hamesha same server pe jaaye (session persistence).

SECTION 09

Databases — SQL vs NoSQL

SQL (PostgreSQL, MySQL)	NoSQL (MongoDB)
Tables mein data rakhta hai — jaise Excel sheet. Rows aur columns. Relationships hain (users table + photos table = foreign key). Structured data ke liye best. Banking, e-commerce mein use hota hai. ACID compliant — data kabhi galat nahi hoga.	**JSON documents mein data rakhta hai** — flexible, har document alag ho sakta hai. Schema fix nahi. Fast read/write. Social media, chat apps, real-time data ke liye best. Horizontally scale karna asaan hai.

EXAMPLE — Photo-Sharing App

SQL use karo in ke liye: User accounts (naam, email, password) — ye structured hai. Payment records — ye 100% accurate hona chahiye. Followers/Following relationships. **NoSQL use karo in ke liye:** Photo metadata (tags, filters, location vary karti hai). Activity feed (har user ka alag structure). Comments aur reactions (flexible schema chahiye). **Video mein yehi approach use hua:** User data ke liye SQL, photos/posts ke liye NoSQL.

KEY POINT: Koi ek sahi nahi hai — dono apni jagah best hain. Real apps mein DONO saath use hote hain (Polyglot Persistence).

SECTION 10

Blob Storage — Photos/Videos Ka Godown

Blob (Binary Large Object) Storage bari files store karne ke liye hota hai — photos, videos, PDFs. Ye database mein nahi rakhte kyun ke database slow ho jaata hai bari files se. Alag dedicated storage use hota hai.

EXAMPLE — Photo-Sharing App — Core Concept

Ye video ka SABSE important concept hai kyun ke hum photo-sharing app bana rahe hain! User photo upload karta hai --> Photo **Blob Storage (AWS S3)** mein jaati hai --> Database mein sirf photo ka **URL/link** save hota hai. Socho Google Drive hai tera — photos wahan rakhta hai. Database mein sirf 'ye photo is link pe hai' likhta hai. Isse database chhota aur fast rehta hai. **Tools:** AWS S3, Google Cloud Storage, Cloudflare R2, Azure Blob Storage.

KEY POINT: KABHI bhi bari files (images, videos) database mein mat rakho. Blob Storage use karo aur database mein sirf URL rakho.

SECTION 11

CDN — Content Delivery Network

CDN static files (images, CSS, JS, videos) ko duniya bhar ke servers pe copy kar deta hai taa ke user ko sabse qareeb wala server se milein. Tera main server Islamabad mein hai — lekin USA ka user ko USA se serve hoga.

```
User (Lahore) --> CDN Server (Lahore) --> Photo milti hai (FAST!) User (New York) --> CDN Server (New York) --> Same photo milti hai (FAST!) Dono ko Islamabad ke main server tak jaane ki zaroorat NAHI
```

EXAMPLE — Photo-Sharing App

Ali ne Islamabad se ek photo upload ki. Woh photo blob storage mein gayi. Ab jab Ahmed (Karachi), Sara (Dubai), aur John (London) ye photo dekhein ge — unhe Islamabad ke server se nahi milegi. CDN ne photo ko Karachi, Dubai, aur London ke servers pe already copy kar di hogi. **Result: Sab ke liye fast loading. Tools:** Cloudflare, AWS CloudFront, Vercel Edge, Fastly.

KEY POINT: CDN = Pizza chain ki branches. Agar sirf ek branch ho (Islamabad) toh doosre shehron ko wait karna padega. Har shehr mein branch = CDN edge server.

SECTION 12

Caching — Speed Ka Raaz

Caching ka matlab hai: jo data baar baar chahiye, usse database se har dafa nikalke lane ki bajaye kahi qareeb (memory mein) rakh lo. Isse response **10x-100x fast** ho jaata hai.

```
Request aaya --> Cache mein hai? --> YES --> Direct return (2ms) --> NO --> DB se lao (200ms) --> Cache mein daalo --> Return
```

EXAMPLE — Photo-Sharing App

Bina Cache: Har dafa jab koi Explore page kholata hai, database mein query: 'top 50 trending photos lao'. Ye heavy query hai, 200ms lagti hai. 1 lakh log ek minute mein kholein toh database mar jaaye ga. **Cache ke saath:** Pehli dafa query ki, result Redis cache mein rakh diya (5 minute ke liye). Ab agle 1 lakh requests Redis se 2ms mein jawab mil jaaye ga. Database ko ek query bhi nahi lagti. Jaise **chai ka thermos** — subah bana ke rakh li, poora din bina dubara banaye milti rehti hai.

KEY POINT: Cache Rule: Jo data BOHOT zyada read hota hai aur KAM change hota hai — woh cache karo. Tool: Redis ya Memcached.

SECTION 13

Message Queues — Background Kaam

Kuch kaam aise hote hain jo **abhi nahi** karne — lekin hone chahiyein. Jaise email bhejna, notification dena, photo resize karna, video encode karna. Inhe ek queue (line) mein daal do, background mein ho jayenge. User ko wait nahi karna padega.

```
User ne photo upload ki --> API: 'Saved!' (instant response) --> Queue mein: 'Resize karo, thumbnail banao, CDN pe push karo, followers ko notify karo' --> Workers ek ek karke karte hain (background mein)
```

EXAMPLE — Photo-Sharing App — Real Use Case

Ahmed ne photo upload ki. Agar sab kuch synchronously karo: Photo save (1s) + Resize 5 sizes (3s) + Thumbnail (1s) + CDN push (2s) + Notify 500 followers (5s) = 12 seconds wait! Message Queue ke saath: Photo save (1s) --> User ko 'Done!' --> Baaki sab queue mein chala gaya, background mein hota rehta hai. **User experience: 1 second. Baqi kaam peeche chalta rehta hai.** Jaise **restaurant ki order slip** — waiter slip laga ke agla order le leta hai. Kitchen apni speed se banata hai.

KEY POINT: Tools: RabbitMQ, Apache Kafka, AWS SQS, BullMQ (Node.js). Rule: Agar kaam time-sensitive nahi hai, queue mein daalo.

SECTION 14

Scaling — Horizontal vs Vertical

Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Ek hi server ko **zyada powerful** bana do — zyada RAM, CPU, SSD. Simple hai lekin ek LIMIT hai — ek machine kitni powerful ho sakti hai? Single point of failure bhi hai — ek server crash = sab down.	**Aur servers add karo** — load sab mein divide ho jaaye. Complex hai lekin virtually UNLIMITED hai. Ek server crash ho toh baaki kaam karte rahein — fault tolerant.

EXAMPLE — Photo-Sharing App

Vertical Scaling: Ek server hai jismein 8GB RAM hai. 10,000 users handle kar raha hai. Users barh gaye — RAM 32GB kar di. Phir 64GB. Phir 128GB. Ab aur nahi ho sakti — LIMIT! **Horizontal Scaling:** 3 servers lagao, har ek 8GB RAM. 30,000 users handle. Users aur barh gaye? 5th server add karo. 10th add karo. Aur karo. **Koi limit nahi.** Instagram, Netflix, Facebook — sab **horizontally scale** hote hain. Isliye ye kabhi down nahi hote.

KEY POINT: Senior engineers HAMESHA horizontal scaling ke liye design karte hain. Vertical ek temporary fix hai, horizontal permanent solution.

SECTION 15

Replication — Database Ki Photocopy

Replication ka matlab: ek database ki copies bana ke rakh lo. Ek **Primary (Master)** hai jahan writes hote hain. Ek ya zyada **Replicas (Slaves)** hain jahan se reads hote hain. Agar primary crash ho jaaye toh replica take over karti hai.

```
Writes (photo upload) --> Primary DB Reads (feed scroll) --> Replica 1, Replica 2, Replica 3 Primary crash? --> Replica 1 promote to Primary (automatic failover)
```

EXAMPLE — Photo-Sharing App

Teri app pe 90% kaam READ hai (log photos dekhte hain) aur sirf 10% WRITE hai (upload/like/comment). 1 Primary DB: Sirf writes handle kare (photo save, like save). 3 Replica DBs: Reads handle karein (feed dikhao, profile dikhao, search karo). Ab 90% load 3 servers pe divide ho gaya. Primary pe sirf 10% load. Aur agar Primary crash ho jaaye — Replica 1 turant Primary ban jaaye gi. **Downtime = Zero.**

SECTION 16

Sharding — Data Ko Todh Do

Jab tera data itna barh jaaye ke **ek database mein fit nahi hota**, toh usse todh ke multiple databases mein baat do. Yehi **Sharding** hai.

EXAMPLE — Photo-Sharing App

50 million users hain. Ek database slow ho raha hai. **Sharding Strategy — User ID se:** Shard 1: Users 1 - 10M (aur unki photos, likes) Shard 2: Users 10M - 20M Shard 3: Users 20M - 30M Shard 4: Users 30M - 40M Shard 5: Users 40M - 50M Ab jab User #25M ki photo dekhni hai, seedha **Shard 3** pe jaao. Baqi 4 databases ko touch bhi nahi karna. Jaise **library ki shelves** — A-F ek shelf, G-L doosri. 'Python' book chahiye toh seedha P wali shelf pe jaao.

KEY POINT: Sharding mushkil hai (joins complex ho jaate hain, resharding pain hai) lekin bade scale pe ZAROORI hai. Instagram aur Twitter sab sharding use karte hain.

SECTION 17

CAP Theorem — Teen Mein Se Do Chuno

Distributed systems ka sabse important rule. Kehta hai ke teen cheezein hain — **tum sirf DOO choose kar sakte ho**, teeno ek saath mumkin nahi:

C — Consistency: Har read pe LATEST data mile. Purana data kabhi na dikhe. **A — Availability:** Har request ka JAWAB aaye — system kabhi 'down' na bole. **P — Partition Tolerance:** Network toot jaaye toh bhi system kaam kare. Distributed systems mein P hamesha chahiye. Toh asli choice **C aur A ke beech** hoti hai.

EXAMPLE — Photo-Sharing App

Photo Feed (AP — Availability + Partition Tolerance): Agar kisi ko photo 2 second late dikhe — koi masla nahi. Lekin app hamesha kaam kare. Isliye feed ke liye Availability choose karte hain. **Like Count (CP — Consistency + Partition Tolerance):** Agar ek photo pe 1000 likes hain aur kisi ko 500 dikhein — masla hai. Isliye like count ke liye Consistency choose karte hain. Thoda slow hoga, lekin galat nahi hoga. **Same app mein ALAG components ke liye ALAG choice ho sakti hai!**

SECTION 18

Microservices vs Monolith

Monolith	Microservices
Poora app **ek codebase, ek server**. Simple, fast development. Choti teams ke liye perfect. Lekin ek bug poora app crash kar sakta hai. Deploy = poora app deploy. Jaise **ek ghar** — sab kuch ek structure mein.	Har feature **alag service** hai — apna server, apna database. Independent deploy, independent scale. Complex lekin bohot scalable. Jaise **apartment building** — har flat independent. Ek mein masla ho toh baaki sab theek.

EXAMPLE — Photo-Sharing App

Monolith (shuru mein): Ek Node.js app — photo upload, user auth, feed, notifications, sab ek mein. **Microservices (bade scale pe):** - Photo Service: Upload, resize, store - User Service: Signup, login, profile - Feed Service: Timeline generate karo - Notification Service: Push notifications - Search Service: Photos/users search karo Har service independently scale aur deploy ho sakti hai. Photo uploads barh gaye? Sirf Photo Service ke servers barhaao — baaki ko mat chhoo.

KEY POINT: Video ka rule: 'START with monolith, SPLIT when you NEED to.' Pehle din se microservices banana = over-engineering.

SECTION 19

Rate Limiting — Abuse Se Bachaao

Rate Limiting ka matlab: ek user ya IP se kitne requests allowed hain ek waqt mein. Isse system abuse, DDoS attacks, aur bots se bachta hai.

EXAMPLE — Photo-Sharing App

Bina Rate Limiting: Ek bot 1 second mein 50,000 requests bhejta hai teri API pe. Server crash. Asli users ke liye app down. **Rate Limiting ke saath:** - Ek IP se max 100 requests per minute - Ek user max 10 photo uploads per hour - Login attempts: max 5 per 15 minutes Limit cross hone pe: HTTP 429 'Too Many Requests' response. Jaise **buffet ka rule** — ek insaan sirf 2 dafa plate bhar sakta hai. Agar koi 50 dafa jaaye toh baakiyon ke liye kuch nahi bachega.

SECTION 20

WebSockets — Real-Time Connection

Normal HTTP mein client request bhejta hai, server response deta hai — phir connection band. **WebSockets** mein connection **khula rehta hai** — dono taraf se data aa sakta hai bina naye request ke.

HTTP (Normal)	WebSocket
Client: 'Koi naya message hai?' Server: 'Nahi' (2 sec baad) Client: 'Ab hai?' Server: 'Nahi' (2 sec baad) Client: 'Ab?' Ye hai **polling** — bandwidth waste.	Client: 'Connection kholo' Server: 'Khol diya' (silence...) Server: 'Naya message aaya!' Server: 'Aur ek aaya!' Ye hai **real-time** — efficient.

EXAMPLE — Photo-Sharing App

HTTP Polling: App har 5 second mein server se poochta hai 'koi naya like aaya?' — bandwidth waste. **WebSocket:** Connection khula hai. Jaise hi koi like kare, server turant batata hai — instant notification. Real-time features ke liye — likes, comments, typing indicators, live stories — **WebSockets zaroori hain.** Socho **phone call vs SMS**. SMS mein har dafa type karo, bhejo, wait karo. Phone call mein line khuli hai — jab ji chahe bolo.

SECTION 21

Trade-offs & Summary — Final Khulaasa

System design mein **koi PERFECT answer nahi hota**. Har decision mein kuch milta hai aur kuch jaata hai. Senior engineers ka asli skill yehi hai — **samajhna ke kis cheez ke badle kya chhod rahe ho** aur **KYUN**.

Concept	Ek Line Mein	Photo App Mein
Client-Server	Maangne wala + dene wala	Phone = client, server = photos deta hai
DNS	Naam se IP dhoondho	photoshare.com --> IP address
API	System ka darwaza	GET /photos, POST /upload
Load Balancer	Traffic baanto	5 servers mein requests divide
Database	Data rakho	SQL users, NoSQL photos
Blob Storage	Bari files rakho	S3 mein photos, DB mein sirf URL
CDN	Content qareeb rakho	Lahore user = Lahore CDN server
Cache	Baar baar ka data yaad rakho	Redis mein trending photos
Queue	Background kaam	Photo resize, notifications
Scaling	Zyada load handle karo	Servers add karo (horizontal)
Replication	DB ki copy	3 read replicas, 1 primary
Sharding	DB todh do	Users 1-10M shard 1, 10-20M shard 2
CAP	2 mein se choose karo	Feed=AP, Likes=CP
Microservices	Features todh do	Photo svc, User svc, Feed svc
Rate Limiting	Zyada requests roko	100 req/min per IP
WebSockets	Real-time connection	Instant like notifications

KEY POINT: System Design = Sahi sawaal poochna + Bade blocks mein sochna + Trade-offs samajhna + Failure ke liye ready rehna. Code last mein aata hai, SOCH pehle aati hai.